P4. GESTIÓN DE DATOS SANITARIOS CON MEMORIA VIRTUAL

Sistemas Operativos 1
Mayo 2026

1 INTRODUCCIÓN

En los sistemas informáticos modernos, la gestión eficiente de grandes volúmenes de datos es fundamental, especialmente en ámbitos como la salud pública, donde se analizan registros históricos para estudiar la evolución de indicadores sanitarios. Para ello, los sistemas operativos proporcionan mecanismos avanzados de gestión de memoria que permiten trabajar con grandes conjuntos de datos de forma eficiente.

Entre estos mecanismos destacan las funciones de **asignación dinámica de memoria** (`malloc`, `realloc`, `free`) y el uso de **memoria virtual mapeada en ficheros** mediante `mmap()`, que permite acceder a grandes archivos como si fueran parte del espacio de direcciones del proceso.

La temática de esta práctica se enmarca en el **Objetivo de Desarrollo Sostenible 3 (ODS 3: Salud y bienestar)**, que promueve la mejora de la salud global mediante el análisis y seguimiento de indicadores sanitarios como la esperanza de vida.

2 DESCRIPCIÓN DE LA PRÁCTICA

En esta práctica se desarrollará un sistema capaz de **leer, almacenar y analizar datos históricos de esperanza de vida** a partir de un fichero CSV que contiene registros de distintos países y años. Cada registro representa un país, su código internacional, un año y el valor de esperanza de vida correspondiente a ese año.

El programa deberá **implementar dos modos de lectura de datos**: uno utilizando memoria dinámica mediante `malloc/realloc` y otro utilizando memoria virtual mediante `mmap()`. Tras cargar los datos, el programa deberá calcular estadísticas básicas y permitir consultas por año.

El objetivo es comprender cómo funcionan los mecanismos de memoria virtual y cómo afectan al rendimiento del sistema.

3 ESTRUCTURA DEL DATASET

Se utilizará el fichero `life_expectancy.csv`, obtenido de la web https://ourworldindata.org/, que contiene aproximadamente 21.500 registros. Se provee también de un fichero `life_expectancy_small.csv` con 1.000 registros para *test*/*degugging*. Ambos ficheros contienen líneas con información como la que se muestra a continuación:

```
Entity,Code,Year,Life expectancy (years)
Spain,ESP,1990,76.4
France,FRA,1990,77.1
Japan,JPN,1990,78.9
...
```

Se deberá utilizar la siguiente estructura obligatoria:

```
typedef struct {
    char country[64];
    char code[8];
    int year;
    float life_expectancy;
} LifeRecord;
```

4 ARGUMENTOS DE ENTRADA

Para facilitar la claridad del programa, se pide utilizar la siguiente sintaxis en los argumentos de entrada.

```
./practica4 <modo> <archivo> [sort]
```

Dónde `modo` puede tomar dos valores:

- `malloc` → lectura dinámica
- `mmap` → lectura con memoria virtual

`sort` (opcional) permite la ordenación de los resultados por año.

5 DETALLES DE LA IMPLEMENTACIÓN

Para la lectura de datos con `malloc/realloc`:

1. Reservar memoria inicial para 100 registros.
2. Leer cada línea mediante `fgets`.
3. Parsear datos usando `strtok`.
4. Si la memoria se llena, duplicar la capacidad mediante `realloc`.
5. Guardar los registros en memoria.
6. Continuar hasta finalizar el fichero.

Por otro lado, la lectura con `mmap` requiere:

1. Abrir fichero con `open()`.
2. Obtener tamaño con `fstat()`.
3. Mapear fichero con `mmap()`.
4. Recorrer memoria detectando saltos de línea.
5. Parsear cada línea.
6. Liberar memoria con `munmap()`.

Además, se pide implementar una funcionalidad opcional que permita ordenar los registros por año utilizando `qsort()` y mostrar los primeros 10 registros.

6 EJEMPLOS DE SALIDA

```
./practica4 malloc life_expectancy.csv
Total registros: 21565
Media global: 58.60 años
Mayor valor: Monaco (2023) -> 86.37 años
Menor valor: Americas (34) -> 0.00 años

Introduce un año: 2000
Media en 2000: 68.06 años
```

```
./practica4 malloc life_expectancy.csv sort
Total registros: 21565
Media global: 58.60 años
Mayor valor: Monaco (2023) -> 86.37 años
Menor valor: Americas (34) -> 0.00 años

Introduce un año: 2000

Media en 2000: 68.06 años
Primeros 10 registros:
Americas 34 0.00
Americas 34 0.00
Americas 35 0.00
Americas 35 0.00
Least developed countries 36 0.00
Least developed countries 36 0.00
England and Wales 37 0.00
Least developed countries 37 0.00
Least developed countries 37 0.00
```

7 VALIDACIÓN CON VALGRIND

Valgrind es una herramienta de línea de comandos utilizada para detectar errores relacionados con el uso de memoria en programas, especialmente en lenguajes como C y C++. Es especialmente útil en programas en C, donde el programador debe gestionar manualmente la memoria con funciones como `malloc`, `realloc` y `free`. Con Valgrind es posible detectar errores como fugas de memoria, accesos fuera de los límites de un bloque o uso de memoria no inicializada. En esta práctica se utilizará para verificar que el programa libera correctamente toda la memoria utilizada y no contiene errores relacionados con su gestión.

Para ejecutar valgrind, se utiliza un comando como el que se muestra a continuación.

```
valgrind --leak-check=full ./practica4 malloc life_expectancy.csv
```

Se espera un análisis detallado de la herramienta indicando que no existen fugas de memoria. En caso de detectar fugas, éstas se deberán corregir.

8 PREGUNTAS GENERALES

P1. Explica la diferencia entre `malloc` y `mmap` en cuanto a cómo se reserva memoria y quién la gestiona. Indica una ventaja y un inconveniente de cada método.

P2. Explica qué función de la librería estándar has utilizado para separar los campos de la línea y qué carácter has utilizado como delimitador.

P3.¿Qué función estándar has utilizado para ordenar los registros y qué parámetros recibe?

P4. ¿Por qué es necesario utilizar `realloc` en esta práctica? Describe en qué situación concreta se produce su uso durante la ejecución del programa.

P5. ¿Qué ocurre si se olvida liberar la memoria reservada con `free`? ¿Cómo puede detectarse este problema utilizando valgrind?

P6. ¿Qué ventaja tiene utilizar `mmap` frente a leer el fichero línea a línea con `fgets` cuando el fichero es muy grande?

P7. ¿Qué estructura de datos has utilizado para almacenar los registros y por qué es necesario controlar su capacidad?

P8. ¿Qué ocurre si el fichero CSV está vacío o contiene solo la cabecera? Describe cómo debería comportarse tu programa

9 ENTREGA Y EVALUACIÓN

Se pide entregar un único **fichero en C** (`practica4.c`) implementando la funcionalidad descrita anteriormente. Además se requiere la entrega de un informe sobre el trabajo realizado y con respuestas a las preguntas planteadas. **El informe se debe entregar en formato PDF** (NO se admiten informes en formato doc, docx, odt, etc). Este informe debe incluir una **introducción**, una sección donde se muestran y comentan las **pruebas realizadas y respuestas a las preguntas**, así como una sección con las **conclusiones y valoraciones personales de la práctica.** Importante: **se ha de indicar en una sección la contribución de cada uno de los miembros del grupo, indicando el porcentaje de la nota que merecería cada uno (p.ej., 25%-25%-50%; 50%-50%; 30%-70%).**

El documento ha de tener un **máximo de 6 páginas** (sin contar la portada o índice). Se evaluará las pruebas realizadas (60%), escritura y expresión (10%), así como la estructura y presentación del informe (30%).

Cada grupo debe subir un **único archivo ZIP** a la tarea asociada a esta práctica en el campus virtual con el nombre **P4_GrupoXX_nombre1_apellido1_nombre2_apellido2.zip**, y debe incluir el **fichero C** y el **informe** solicitado con todas las secciones especificadas en el apartado anterior.

La nota final de la práctica tendrá los siguientes pesos: 70% código C y 30% informe.